

COMMON PLATFORM · HM COURTS & TRIBUNALS SERVICE

RAG Service

Grounded answers over court documents, as a service

A guide for service owners, delivery teams and assurance functions considering the CP AI RAG Service for their own users.

Contents

1	Executive summary	3
	What this document answers · Who should read what · In one paragraph	3

2	What the service does	4
	The problem it addresses · The capability set · How evidence is refined before the model sees it	4

3	Built as a service	6
	The principle · The shape of it · Five functions, one contract · Processing is decoupled from serving	6

4	Connecting to the service	7
	Three steps, and no more · What you never hold · Client identity	7

5	Consuming the service	8
	The document workflow · Asking questions · Sync or async · Limits worth designing against	8

6	Trust, assurance and security	10
	Citations, enforced · Measured in production · Security posture · Data handling	10

7	Applying it in your service	12
	Three patterns of adoption · What is ready, and what is not · What we would need from you	12

8	Reference	13
	Endpoints · Glossary · Where to read next · Contact	13

Executive summary

The CP AI RAG Service lets Common Platform systems upload court documents and ask questions of them, getting back grounded answers in which **every claim is cited to a document and a page**. It is a fully serverless Azure pipeline consumed entirely over HTTP: no SDK to adopt, no infrastructure to run, no model credentials to hold.

The service is deliberately a **capability, not a product**. It has no user interface of its own. You bring the screen and the users; the service brings extraction, indexing, retrieval, grounded generation, and a measured trust story underneath.

5

HTTP operations behind one published, linted OpenAPI contract

2

invocation modes — one round trip, or request-and-poll

100%

of production answers scored for groundedness by a judge model

0

storage keys, connection strings or model credentials change hands

The consequence is the argument of this document: **trust in the answers is not a policy statement — it is enforced in the pipeline and measured in production**. A citation guard means an ungrounded answer is never returned silently, and every answer that is returned is scored.

1.1 What this document answers

QUESTION	WHERE IT IS ANSWERED
What does the service actually do?	Section 2 — What the service does
How do I authenticate against it?	Section 4 — Connecting to the service
Can I trust what it produces?	Section 6 — Trust, assurance and security
What would adopting it involve for me?	Section 7 — Applying it in your service
What is not built yet?	Section 7.2 — What is ready, and what is not

1.2 Who should read what

IF YOU ARE	READ
A senior responsible owner	Sections 1, 2 and 7. Scoped as grounded question answering — decision support, never decision automation.
A service owner in another area	Sections 2, 4 and 7. You consume a capability, not a product: scope it with your own metadata, present it in your own interface.
A delivery or engineering team	Sections 3 to 5, then the endpoint summary in Section 8. Five operations, one contract-first OpenAPI spec.
Security, IG or assurance	Section 6 in full. Entra ID at the edge, managed identity throughout, groundedness measured on every answer.

1.3 In one paragraph

You request an upload link, put a PDF straight into storage, and the service extracts, chunks, embeds and indexes it page by page. A question runs hybrid retrieval scoped by your metadata and returns an answer in which every claim points to a document and a page, guarded and scored. You bring a user and an interface; we keep the data, the models and the credentials.

SECTION 2

What the service does

2.1 The problem it addresses

Case material accumulates faster than anyone can read it. The answer to a specific question — what did the witness actually say about the vehicle, which conditions were attached, what does the guidance require — is somewhere in the documents, but finding it is slow work, and a wrong recollection is not a small thing.

Generic chat tools answer fluently whether or not the material supports them. That failure mode is precisely the one a justice system cannot absorb. The RAG Service exists to make the opposite trade: **an answer you can check beats an answer that merely sounds right.**

*Every claim points to a document and a page,
or the answer does not pass.*

ENFORCED BY THE CITATION GUARD, NOT ASSERTED IN POLICY

Everything the service writes is grounded in retrieved passages from your own documents. The model must emit a structured fact map alongside its prose; a guard checks it; a judge model scores the result. Nothing about that is optional or bolted on — it is the shape of the pipeline.

2.2 The capability set

Six capabilities, each reachable as an HTTP call. Most integrations begin with two: upload, then ask.

TWO-STEP · DIRECT TO STORAGE

Document upload

Request a short-lived, single-blob SAS link and PUT the PDF straight to storage. File bytes never pass through the API tier. Duplicates rejected; your metadata attached to every chunk.

POST /document-upload

ASYNCHRONOUS · QUEUE-DRIVEN

Ingestion pipeline

Document Intelligence layout extraction, per-page chunking, embeddings, and indexing into Azure AI Search — hybrid vector plus keyword, with page-level provenance kept throughout.

automatic on upload

READ

Ingestion status

A status row per document from the moment the upload link is issued: awaiting upload, awaiting ingestion, success, failure, or over the size limit.

GET /document-upload/{documentReference}

SYNCHRONOUS

Grounded answers, one round trip

Ask a question scoped by metadata; the answer, its citations and the evidence chunks come back in the same response.

POST /answer-user-query

ASYNCHRONOUS · REQUEST AND POLL

Grounded answers, long-running

The same pipeline behind a transaction ID and a polling endpoint, for interfaces that must not hold a request open.

POST /answer-user-query-async

MEASURED · EVERY ANSWER

Groundedness scoring

A separate judge model rates every production answer 1–5 against the retrieved evidence; scores land in Azure Monitor with per-client dimensions.

automatic on every answer

2.3 How evidence is refined before the model sees it

Retrieval over-fetches a candidate pool of fifty chunks — vector nearest-neighbour and keyword search in one call, filtered to your metadata. Three stages then shape what the model actually reads. Each is independently configurable; together they are why answers stay dense without losing the rare fact that matters.

1 Containment dedup — information-safe by construction

Court material is full of passages duplicated across files: the same order quoted in three bundles, the same paragraph in a covering letter and its enclosure. This stage drops a chunk **only** when nearly all of its content already appears in a higher-ranked chunk. A copy that carries one extra crucial sentence is not a duplicate, and survives. Cross-file copies collapse; unique information never does.

2 Semantic dedup — available, off by default

A coarser cosine-similarity stage exists but ships disabled: it can drop a near-duplicate that carries unique content, which is exactly the failure the containment stage was built to avoid. It remains a toggle, not a default.

3 MMR diversification — a balanced evidence set

Maximal Marginal Relevance selects the final fifteen chunks, balancing relevance against redundancy. The model reads a smaller, more varied evidence set — fewer tokens, better coverage of the question's aspects.

2.4 What an answer contains

An answer is not a paragraph of prose. The response carries the answer text with inline, human-readable source references; the full set of retrieved evidence chunks (document, page, content, your metadata) so your interface can show its working; and the query and prompt echoed back with generation timing. The citations arrive machine-readable before they are prose: the model emits a structured fact map — citation ID, document, file name, page numbers — which the service resolves, merges and rewrites into the readable form.

IN THE PAYLOAD	WHY IT IS THERE
Answer text with inline citations	The prose your users read; every claim carries its source and page.
Evidence chunks , optionally	Display the passages beside the answer, or keep them for audit. Nothing is asserted that you cannot show.
Echoed query & prompt	The request is reproducible: what was asked, how the task was framed, when, and how long it took.
Status & reason (async)	Pending, generated or failed — with the reason recorded when an answer was delivered degraded.

THE HONEST CAVEAT

The service is single-turn: each question stands alone, with no conversation memory and no streaming. It is PDF-first, and chunk sizing is fixed service-side. These are current scoping decisions, stated here so you can design against them rather than discover them.

SECTION 3

Built as a service

3.1 The principle

The service is **contract-first**. A published OpenAPI specification — linted, versioned and released from its own repository — is the source of truth for every request and response shape; the service's own models are generated from it. Consumers integrate against the contract, not against our implementation, and the contract cannot drift silently because the implementation is derived from it.

3.2 The shape of it

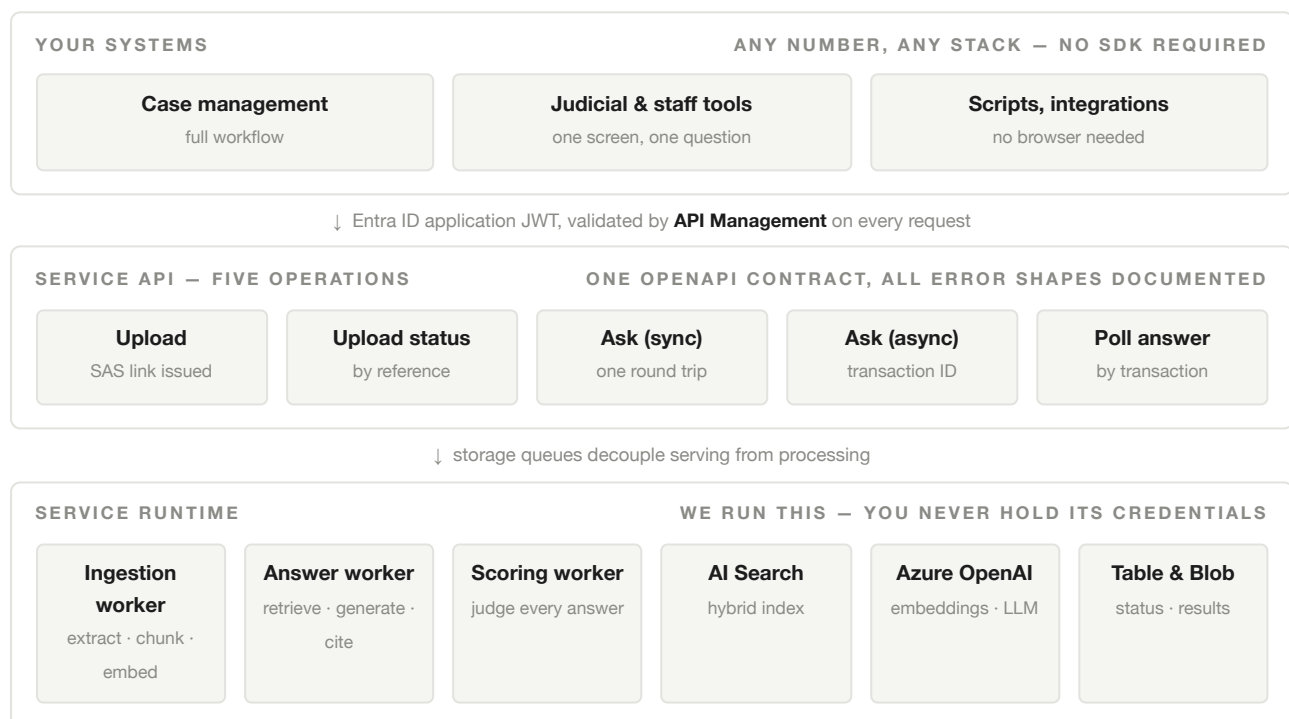


FIGURE 1 The layers of the service. Everything below the first arrow is operated by the Common Platform team. Your responsibility stops at obtaining a token and making a request.

3.3 Processing is decoupled from serving

Every expensive step — extraction, embedding, generation, scoring — runs as a short, retryable, queue-triggered invocation on serverless compute that scales with load. The HTTP surface stays fast because it only ever validates, records and enqueues.

Duplicate deliveries and worker crashes are a designed-for condition, not an incident. A shared idempotency guard keyed on the document or transaction holds a fenced, time-limited lease over in-flight work: a redelivered message for finished work is skipped without re-running models, and a worker resumed after a crash **cannot overwrite a completed answer or double-spend an LLM call**. The final write is fenced by the lease's claim-time ETag — a stale worker's result is discarded, not merged.

SECTION 4

Connecting to the service

4.1 Three steps, and no more

1 Obtain an access token

Acquire an Entra ID application token for the service audience — the standard client-credentials flow every Common Platform consumer already uses.

2 Call through API Management

APIM validates the JWT on every request and injects the function-level credential; the function tier is not reachable around the gateway.

3 Call the operation you need

Five operations, stable paths, every shape in the OpenAPI contract. Point a generator at the spec and you have a typed client in any language.

4.2 What you never hold

This is the part most integration conversations turn on. A consuming system needs none of the following.

- × Storage account keys or connection strings — the platform itself holds none
- × AI model endpoints, keys or deployments
- × A database, or any connection to ours
- × The retrieval pipeline, prompts or trust model
- × Any coordination with a service release
- × Capacity sized for document-scale payloads

4.3 Client identity and isolation

The service is built multi-client: your client identity is threaded through every search filter, storage key, blob name, queue payload and telemetry dimension. Another client's documents are not merely hidden — a cross-client lookup resolves to **404, indistinguishable from a record that never existed**. Document identifiers are unique per client, and isolation is exercised by integration tests, including header-spoofing and cross-client reads.

THE HONEST CAVEAT

Client-identity enforcement is implemented and tested but sits behind a feature flag pending cut-over. Ask us where cut-over stands before you design for co-tenancy.

4.4 A first call, end to end

One ingested document is the only prior state the synchronous endpoint needs — upload-then-ask is the shortest path to proving a connection works.

```
first-answer.sh
```

```
BASH
```

```
curl --fail-with-body \  
-H "Authorization: Bearer $CP_TOKEN" -H "Content-Type: application/json" \  
-d '{ "userQuery": "What conditions were attached to the order?",  
      "metadataFilter": [ { "caseId": "CASE-4417" } ] }' \  
https://<apim-origin>/answer-user-query # → the cited answer + its evidence chunks
```

SECTION 5

Consuming the service

5.1 The document workflow

Three calls take a PDF from a desktop to a question-ready index — one of them direct to storage, so no interface ever handles document-scale bytes.

STEP	CALL	WHAT YOU GET BACK
1	POST /document-upload	A single-blob, short-lived SAS upload URL (create/write/read only; default 120-minute expiry) plus your document reference. Duplicate document IDs are rejected here, before any bytes move.
2	PUT the upload URL	The browser or client uploads the PDF bytes directly to Blob Storage. Files over the size limit are recorded as FILE_SIZE_OVER_LIMIT and never processed.
3	GET /document-upload/{documentReference}	AWAITING_UPLOAD → AWAITING_INGESTION → INGESTION_SUCCESS or INGESTION_FAILED. Poll and render progress; handle the failed state.

At upload you attach your own **metadata** — key/value pairs stored on every chunk and filterable at query time. Metadata is the scoping mechanism: a case ID, a document class, a hearing date. The vocabulary is entirely yours.

New versions are first-class: an upload can name earlier documents it **supersedes**; on ingestion the old chunks are excluded from every search by an always-on filter. No consumer has to remember to exclude stale material.

5.2 Asking questions

STEP	CALL	WHAT YOU GET BACK
A	POST /answer-user-query	Synchronous. The cited answer, the evidence chunks, and timing — in one round trip.
B1	POST /answer-user-query-async	Asynchronous. 202 with a transactionId; the work is queued and durable.
B2	GET /answer-user-query-async-status/{transactionId}	ANSWER_GENERATION_PENDING → ANSWER_GENERATED or ANSWER_GENERATION_FAILED, with the answer payload once ready — optionally including the retrieved chunks (withChunkedEntries).

Every question carries the **user query**, a **query prompt** (your task framing — who the answer is for, what register it takes), and the **metadata filter** scoping retrieval to your documents. You steer the framing; the grounding rules are not yours to relax.

5.3 Sync or async

Synchronous, for interactive screens

One request, one cited answer — for a user waiting on the page, within your timeout budget.

Asynchronous, for everything else

Queued, durable, idempotent under retry — for callers that must never hold a request open.

5.4 Limits worth designing against

LIMIT	DEFAULT	WHAT TO DO ABOUT IT
Document upload size	80 MiB	Over-limit files are recorded, never processed. Check sizes client-side; split outsized bundles.
Upload link expiry	120 min	Request the link when the user is ready to upload, not when the screen opens.
Metadata pair length	40 chars	Capped, and reserved keys rejected with 400. Design a compact vocabulary.
File type	PDF	One configured upload extension. Convert Office material to PDF at your edge for now.
Evidence sent to the model	15 chunks	Fixed by the refinement pipeline. Sharpen the metadata filter rather than broaden the query.
Answer budget	4,000 tokens	Ample for grounded answers; not a document generator.
Queue redelivery	3 attempts	Async failures retry by redelivery, then surface as a failed transaction with a reason. Handle it; do not re-submit blindly.

5.5 What ingestion does with your document

Worth knowing when you reason about answer quality, because the provenance your users see is established here.

STAGE	HOW IT WORKS
Extraction	Azure Document Intelligence layout analysis over the blob — a long-running operation; page boundaries preserved, empty pages skipped. Scanned material is read, not refused.
Chunking	Recursive splitting applied per page, so every chunk carries a page number and in-page position. Page-level citation is possible because provenance is never lost, not because it is reconstructed.
Embedding	A 3,072-dimension embedding model, batched; the same model embeds documents and queries. Vector dimension is enforced on write.
Indexing	Azure AI Search: HNSW vector index alongside BM25 keyword search, your metadata on every chunk, client identity on every record.

DESIGN NOTE

Retrieval quality tracks metadata discipline. A single-case filter turns the corpus into that case's papers; no filter turns it into everything you ever uploaded. Decide your vocabulary before your first production upload.

SECTION 6

Trust, assurance and security

6.1 Citations, enforced

Answer systems are usually asked to be right. This one is additionally required to be **checkable**, because the reader is making a decision on the strength of it.

The fact map

The model must emit a structured map of its claims — citation ID, document, file name, page numbers — alongside the prose. The service resolves it, merges adjacent references, and rewrites markers into readable sources. Citations are data first, prose second.

The guard

An answer with no fact map, an unparseable one, or all of its markers stripped is **citation-degraded**. Policy decides its fate: deliver it flagged with the reason recorded, or reject it outright. A deliberate "the documents do not contain this" refusal always passes — declining is the correct grounded behaviour, and is never punished.

An ungrounded answer is never returned silently.

THE GUARD'S CONTRACT WITH EVERY CONSUMER

On the asynchronous path a degraded answer is retried afresh before the policy applies. The outcome is always explicit: a clean answer, a flagged answer with its reason in the record, or a failed transaction. Never a quiet degradation.

6.2 Measured in production, not just promised

Every answer, scored online

A separate judge deployment rates every production answer's groundedness 1–5 against the very chunks it was generated from, with reasoning recorded. Scores flow to Azure Monitor as a histogram with query-type and client dimensions — your answers, observable as a distribution, per client, in production.

Every change, evaluated offline

A dedicated harness reruns the real pipeline — embed, search, generate, cite — across prompts × models × queries, reporting citation coverage, verbosity and truncation. Prompt and model changes are justified with numbers from this harness before they ship, not after.

6.3 Security posture

Identity and access. API Management validates an Entra ID application JWT on every request and injects the function credential; the function tier is not reachable around the gateway. With client-identity enforcement on, a missing identity is refused with 401 and every read and write is scoped to the resolved client.

Platform and data. Storage access is managed identity only — **no storage keys or connection strings exist anywhere in the configuration**, and storage triggers use identity-based bindings. Inputs are hardened at the contract: schema-validated shapes, UUID constraints, metadata length caps, reserved-key rejection, and escaping of both search-filter and keyword-query inputs against injection.

A POSTURE DECISION, STATED PLAINLY

Azure OpenAI content filtering is disabled by policy: the corpus is sensitive case material, and a filter that refuses to read it would fail the very users the service exists for. The trade-off is deliberate, and compensated by grounding — the model may only restate what the documents say, cited to the page.

6.4 Reliability under retry

Queue redelivery is how the service heals — which means duplicate delivery is the normal case to design for, not the edge case. Three properties hold under any combination of retries, crashes and races:

- ✓ **Finished work is never redone.** A duplicate delivery for a document or transaction already in a terminal state is skipped without touching the models — no double-spent extraction, embedding or generation.
- ✓ **In-flight work is never raced.** A time-limited lease on the status row, claimed by conditional write, means two workers cannot process the same item concurrently; a duplicate arriving against a live lease steps back and redelivers.
- ✓ **A completed answer is never overwritten.** The final write is fenced by the ETag captured when the lease was claimed. A worker resuming after a crash — whose lease was reclaimed — has its stale result discarded at the write, not merged over the finished one.

Model calls carry configurable retries and per-phase timeouts, tuned so that long generation latency is bounded deliberately rather than cut off by an accidental read timeout.

6.5 Observability

Application Insights runs on every function host with requests excluded from sampling, so the trace of a request is complete, not statistical. Logging is structured throughout. Groundedness scores flow through OpenTelemetry to Azure Monitor, dimensioned by query type and client — quality is a chart someone can watch, alert on, and attribute.

6.6 Engineering practice behind the claims

PRACTICE	WHAT IT LOOKS LIKE HERE
Contract-first API	The OpenAPI 3.0 spec lives in its own repository, is Spectral-linted and released independently; server models are generated from it. All five operations document their error responses.
Testing	Around 105 unit test classes with coverage gates, plus an integration module that boots all five function hosts against real Azure resources created per run — covering upload → ingest → query end to end, idempotency, supersede, negative paths and client isolation.
CI & analysis	Azure Pipelines on every PR with SonarQube quality gates. Java 21, multi-module Maven.
Decisions on record	Architecture decisions and designs are documented in-repo — idempotency, multi-client isolation and retrieval refinement each have a written design, not tribal knowledge.

SECTION 7

Applying it in your service

7.1 Three patterns of adoption

SMALLEST STEP

One question, one screen

Add a single ask-the-documents box to a screen you already run, on the synchronous endpoint. Documents you already hold, one new call, no new infrastructure.

days

MOST COMMON

A case-scoped assistant

Wire upload into your existing document flow with a case-ID metadata scheme, and offer grounded Q&A over each case's papers — async, polled, with the evidence chunks displayed beside every answer.

weeks

LARGEST STEP

A document-intelligence feature

Build a product surface of your own on the full contract — bulk ingestion, supersede-on-reupload, per-answer audit views from the scored, recorded pipeline.

a quarter

In every pattern the backend is unchanged. What differs is how much of the surface you expose — and because scoping lives in **your** metadata vocabulary, the service never needs to understand your domain to serve it.

7.2 What is ready, and what is not

The first column is in production use today. The second is stated plainly so you can plan against it.

READY NOW

- ✓ Five operations behind APIM with a published, linted contract
- ✓ Sync and async answering with citations and evidence chunks
- ✓ Citation guard with explicit degraded-answer policy
- ✓ Groundedness scored on every production answer
- ✓ Idempotent, crash-safe queue workers
- ✓ Multi-client isolation implemented and integration-tested

NOT YET

- × **Isolation cut-over.** Enforcement is behind a feature flag pending platform cut-over.
- × **Beyond PDF.** One upload extension; convert at your edge for now.
- × **Conversation.** Single-turn only — no memory, no streaming.
- × **Hard delete.** Superseded documents are hidden by filter, not erased; a GDPR erasure path is platform work in flight.
- × **Quality beyond groundedness.** Relevance and completeness are not yet scored online.

7.3 What we would need from you

1 An identity and a route through APIM

An Entra ID application for your service and an APIM subscription — the standard Common Platform onboarding, nothing bespoke.

2 A metadata vocabulary

The handful of keys you will scope by — case, document class, date. Agreed before first upload.

3 A decision on invocation mode

Sync for interactive screens, async for everything else, per Section 5.3. It decides your UX shape and timeout budget.

SECTION 8

Reference

8.1 Endpoints

OPERATION	SUCCESS	PURPOSE
POST /document-upload	200 SAS URL + reference	Initiate an upload; returns the direct-to-storage link
GET /document-upload/{documentReference}	200 ingestion status	Track a document from link issued to indexed
POST /answer-user-query	200 cited answer	Synchronous question answering
POST /answer-user-query-async	202 transaction ID	Queue a question for durable processing
GET /answer-user-query-async-status/{transactionId}	200 status / answer	Poll the asynchronous result, optionally with evidence chunks

All operations document 400/401/500 error shapes (404 on the GETs) in the contract; the spec repository is the source of truth.

8.2 Glossary

TERM	MEANING
Chunk	A page-scoped passage of a document — the unit of retrieval and citation
Fact map	The structured claim-to-source map the model must emit with every answer
Citation guard	The check that flags or rejects answers whose fact map fails
Groundedness	Whether an answer's claims are supported by the evidence; judged 1–5
Containment dedup	Removal of chunks whose content a higher-ranked chunk already covers
MMR	Maximal Marginal Relevance — the final relevance-vs-diversity selection

8.3 Where to read next

DOCUMENT	FOR
api-cp-ai-rag (spec repository)	The OpenAPI contract — the source of truth for shapes and clients
rag-service-one-pager.pdf	The decision-maker's brief — scope, cost and scaling, one page
ai-service-orchestration-test/	The end-to-end integration suite — a runnable account of real behaviour

8.4 Contact

The shortest route to a decision is one ingested document and one question — both covered in Sections 4 and 5. Common Platform AI team, HMCTS.